

编译 Cryptopp

Windows

下面介绍 windows 10 系统下安装 Qt 5.12.11 x86 mingw32 编译器对应的 Cryptopp 的编译.

Qt 没有封装加解密算法库(其实有个哈希函数的函数)。鉴于 OpenSSL 函数封装不友好, 我们考虑在 C++上使用一种新的, 并且封装友好的, OOAD 程度更高的加解密算法库 Crypto++。

官网:<https://www.cryptopp.com>

API 参考文档网址:<https://www.cryptopp.com/docs/ref/>

从官网下载好后, 解压, 由于自带了 VC 的项目文件, 所以使用 Visual Studio 部署比较容易, 网上教程也很多。这里我们介绍在 Qt 上的部署问题。主要问题在于使用 mingw 编译该源码。在 linux 上也能很好的解决问题。

进入正题

1. 下载 cryptopp565.zip。解压, 比如解压到 D:\cryptopp
2. 接下来很重要!!! 使用记事本或者其他文本编辑器编辑 D:\cryptopp\fpstest.cpp 文件, 将其中的 每一个 'OutputDebugString' 都 替换 为 'OutputDebugStringA' .
D:\cryptopp\socketft.cpp 文件, 将其中的 每一个 WSAStringToAddress 都 替换 为 WSAStringToAddressA. 以上操作主要是避免宽字节的错误.

3. 删除 D:\cryptopp\GNUmakefile 文件(不然之后的编译会报错)

4. 打开 Windows 控制台, 这里切换目录到 crypto++库目录, 输入如下命令即可

D:

```
cd \d \cryptopp
```

```
qmake -project
```

此时该目录下会产生 cryptopp.pro 文件。绝对路径为 D:\cryptopp\cryptopp.pro

- 5.这里我们使用 QtCreator 打开 cryptopp.pro。也可使用文本编辑器直接打开。

将第一行的

TEMPLATE = app 修改为 TEMPLATE = lib

并且在最后添加一行:LIBS += -lws2_32

6.在控制台中输入下列命令进行编译

qmake

mingw32-make -j4 all

7. 等待编译结束, 一般要几分钟

```
C:\Qt\cryptopp565>mingw32-make -j16 all
mingw32-make -f Makefile.Release all
mingw32-make -f Makefile.Debug all
mingw32-make[1]: Entering directory 'C:/Qt/cryptopp565'
g++ -c -fno-keep-inline-dllexport -O2 -Wall -W -Wextra -fexceptions -mthreads -DUNICODE -D_UNICODE -DWIN32 -DMINGW_HAS_SECURE_API=1 -DQT_DEPRECATED_WARNINGS -DQT_NO_DEBUG -DQT_GUI_LIB -DQT_CORE_LIB -fPIC -I. -I. -I.. \Qt5.12.11\5.12.11\mingw73_32\include -I.. \Qt5.12.11\5.12.11\mingw73_32\include\QtGui -I.. \Qt5.12.11\5.12.11\mingw73_32\include\QtANGLE -I.. \Qt5.12.11\5.12.11\mingw73_32\include\QtCore -Irelease -I.. \Qt5.12.11\5.12.11\mingw73_32\mkspecs\win32-g++ -o release\3way.o 3way.cpp
mingw32-make[1]: Entering directory 'C:/Qt/cryptopp565'
g++ -c -fno-keep-inline-dllexport -g -Wall -W -Wextra -fexceptions -mthreads -DUNICODE -D_UNICODE -DWIN32 -DMINGW_HAS_SECURE_API=1 -DQT_DEPRECATED_WARNINGS -DQT_GUI_LIB -DQT_CORE_LIB -fPIC -I. -I. -I.. \Qt5.12.11\5.12.11\mingw73_32\include -I.. \Qt5.12.11\5.12.11\mingw73_32\include\QtGui -I.. \Qt5.12.11\5.12.11\mingw73_32\include\QtANGLE -I.. \Qt5.12.11\5.12.11\mingw73_32\include\QtCore -Idebug -I.. \Qt5.12.11\5.12.11\mingw73_32\mkspecs\win32-g++ -o debug\3way.o 3way.cpp
g++ -c -fno-keep-inline-dllexport -O2 -Wall -W -Wextra -fexceptions -mthreads -DUNICODE -D_UNICODE -DWIN32 -DMINGW_HAS_SECURE_API=1 -DQT_DEPRECATED_WARNINGS -DQT_NO_DEBUG -DQT_GUI_LIB -DQT_CORE_LIB -fPIC -I. -I. -I.. \Qt5.12.11\5.12.11\mingw73_32\include -I.. \Qt5.12.11\5.12.11\mingw73_32\include\QtGui -I.. \Qt5.12.11\5.12.11\mingw73_32\include\QtANGLE -I.. \Qt5.12.11\5.12.11\mingw73_32\include\QtCore -Irelease -I.. \Qt5.12.11\5.12.11\mingw73_32\mkspecs\win32-g++ -o release\adler32.o adler32.cpp
g++ -c -fno-keep-inline-dllexport -g -Wall -W -Wextra -fexceptions -mthreads -DUNICODE -D_UNICODE -DWIN32 -DMINGW_HAS_SECURE_API=1 -DQT_DEPRECATED_WARNINGS -DQT_GUI_LIB -DQT_CORE_LIB -fPIC -I. -I. -I.. \Qt5.12.11\5.12.11\mingw73_32\include -I.. \Qt5.12.11\5.12.11\mingw73_32\include\QtGui -I.. \Qt5.12.11\5.12.11\mingw73_32\include\QtANGLE -I.. \Qt5.12.11\5.12.11\mingw73_32\include\QtCore -Idebug -I.. \Qt5.12.11\5.12.11\mingw73_32\mkspecs\win32-g++ -o debug\adler32.o adler32.cpp
g++ -c -fno-keep-inline-dllexport -O2 -Wall -W -Wextra -fexceptions -mthreads -DUNICODE -D_UNICODE -DWIN32 -DMINGW_HAS_SECURE_API=1 -DQT_DEPRECATED_WARNINGS -DQT_NO_DEBUG -DQT_GUI_LIB -DQT_CORE_LIB -fPIC -I. -I. -I.. \Qt5.12.11\5.12.11\mingw73_32\include -I.. \Qt5.12.11\5.12.11\mingw73_32\include\QtGui -I.. \Qt5.12.11\5.12.11\mingw73_32\include\QtANGLE -I.. \Qt5.12.11\5.12.11\mingw73_32\include\QtCore -Irelease -I.. \Qt5.12.11\5.12.11\mingw73_32\mkspecs\win32-g++ -o release\algpam.o algpam.cpp
g++ -c -fno-keep-inline-dllexport -g -Wall -W -Wextra -fexceptions -mthreads -DUNICODE -D_UNICODE -DWIN32 -DMINGW_HAS_SECURE_API=1 -DQT_DEPRECATED_WARNINGS -DQT_GUI_LIB -DQT_CORE_LIB -fPIC -I. -I. -I.. \Qt5.12.11\5.12.11\mingw73_32\include
```

8. 之后编译完成后, 我们需要拷贝生成的文件

D:\cryptopp552\debug\cryptopp565.dll 不需要动态库的引用库

需要所有头文件拷贝到一个文件夹

9. 这样就得到了编译好的库

Linux

接下来, 介绍在 ubuntu 1604 版本 x64 系统环境下, 编译安装 Cryptopp

sudo apt-get install libcrypto++-dev 完成安装

头文件在 /usr/local/include/cryptopp 中

链接时需要的共享库为 `-lcryptopp`

如果使用 Qt 进行编译 那么需要添加 `LIBS += -lcryptopp`

头文件包含的时候

一般使用

```
#include <cryptopp/aes.h>
```

```
#include <cryptopp/modes.h>
```

```
#include <cryptopp/filters.h>
```

```
#include <cryptopp/hex.h>
```

以上是 AES 加解密需要的头文件

```
#include <cryptopp/rsa.h>
```

```
#include <cryptopp/osrng.h>
```

```
#include <cryptopp/base64.h>
```

以上是 RSA 加解密需要的头文件

实践

依赖的路径和动态库如下所示

```
INCLUDEPATH += $$PWD/cryptopp565-release/include
```

```
LIBS += -L$$PWD/cryptopp565-release/bin/ -lcryptopp565
```

已经写好了封装, 大家可以直接使用. 详情看课上的演示